COL 1101

LECTURE - 7

AUGUST 6, 2026

INHERITENCE, METHOD RESOLUTION ORDER

SEQUENCE ADT, ARRAY

WARM-UP : BEFORE THE CLASS

```
def collect(x, box=[]):
    box.append(x)
    return box
```

```
print(collect(1))
print(collect(2))
```

option 1 : [2]

option 2 : [1, 2]

What is the output ?

LAST CLASS: INHERITENCE IN OOP

Two types

1. interface inheritence - Abstract Base Class

2. implementation inheritence - for code reuse

should be used carefully!

Implementing Set ADT Using List

```
class ListSet:
    def __init__(self):
        self._items = []

    def add(self, x):
        if x not in self._items:
            self._items.append(x)

    def contains(self, x):
        return x in self._items

    def remove(self, x):
        if x not in self._items:
            raise KeyError(x)
        self._items.remove(x)

    def size(self):
        return len(self._items)
```

TEMPTING : A SET IS-A LIST

```
class ListSet:
    def __init__(self):
        self._items = []

    def add(self, x):
        if x not in self._items:
            self._items.append(x)

    def contains(self, x):
        return x in self._items

    def remove(self, x):
        if x not in self._items:
            raise KeyError(x)
        self._items.remove(x)

    def size(self):
        return len(self._items)
```



```
class ListSet(list):
    def add(self, x):
        if x not in self:
            self.append(x)
```

def size(self):
    len(self)

IS-A OPENS EVERY DOOR.

```
class ListSet(list):
    def add(self, x):
        if x not in self:
            self.append(x)
```

Every list operation is a fair game!

s = ListSet()

s.append(2)

s.insert(0,3)

s[0] = 3

IS-A vs HAS-A

HAS-A : composing classes

```
class ListSet:
    def __init__(self):
        self._items = []

    def add(self, x):
        if x not in self._items:
            self._items.append(x)

    def contains(self, x):
        return x in self._items

    def remove(self, x):
        if x not in self._items:
            raise KeyError(x)
        self._items.remove(x)

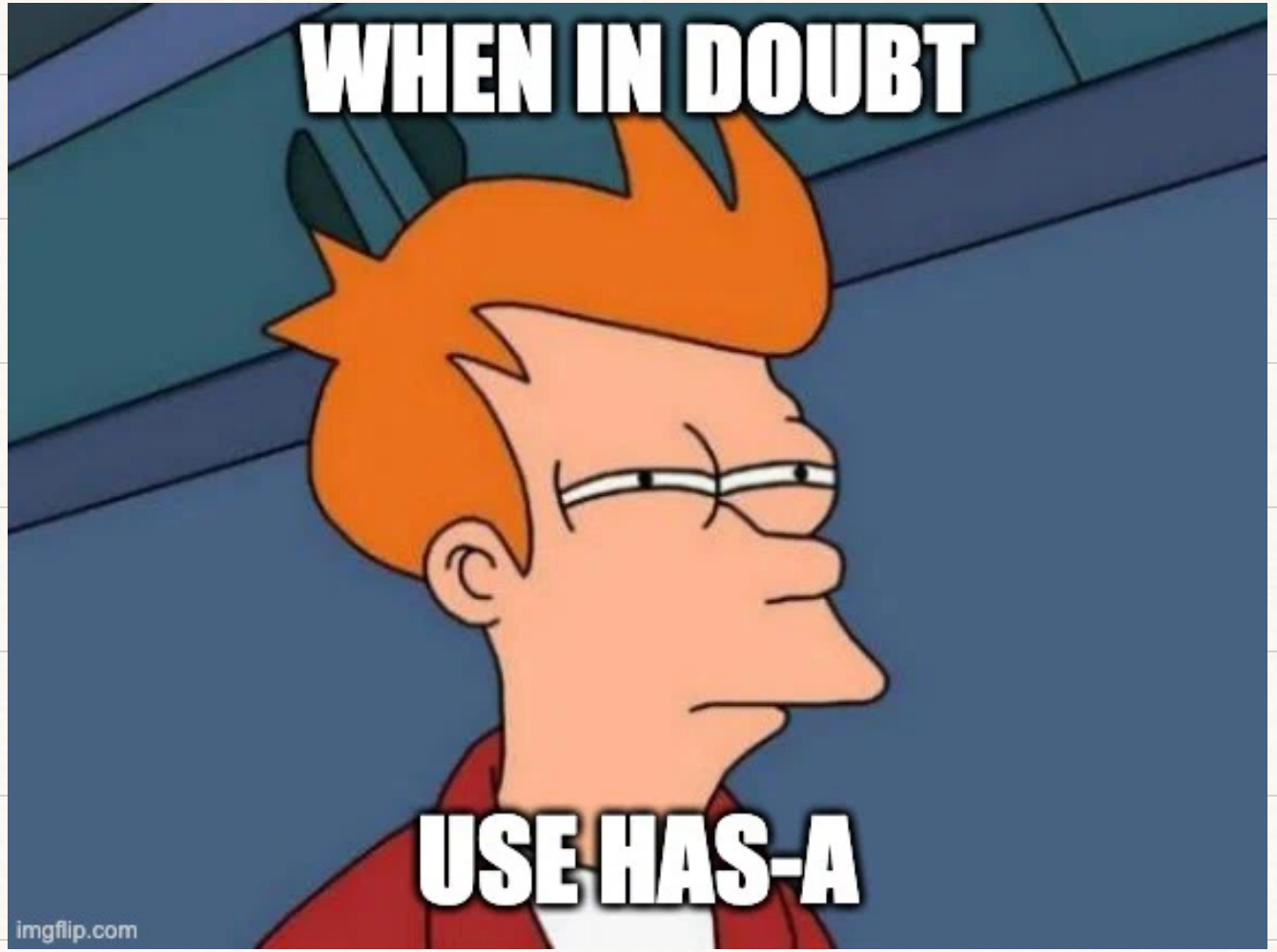
    def size(self):
        return len(self._items)
```

a Set HAS a list

IS-A

```
class ListSet(list):
    def add(self, x):
        if x not in self:
            self.append(x)
```

a Set IS everything a list is.


WHEN IN DOUBT
USE HAS-A
imgflip.com

WHEN MAY YOU INHERIT? LSP

Liskov Substetution Principle: If class B is a subclass of class A, one should be able to use an object of class B in place of an object of class A without breaking the program

(necessary but not sufficient)

A VALID IS-A

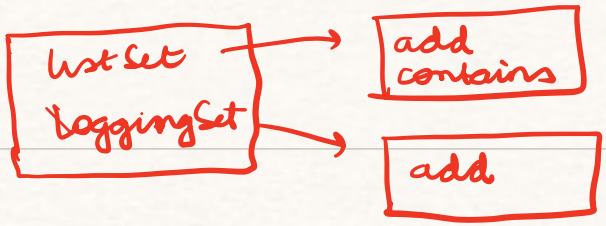
A Set that logs each add, else unchanged

```
class LoggingSet(ListSet):
    def add(self, x):
        print(f"adding {x!r}")
        super().add(x)
```

How Is THE METHOD FOUND (IN PYTHON)?

```
class LoggingSet(ListSet):
    def add(self, x):
        print(f"adding {x!r}")
        super().add(x)
```

```
s = LoggingSet()
s.add(1)
print(s.contains(1))
```



class → parent

How Is THE METHOD FOUND (IN PYTHON)?

```
class LoggingSet(ListSet):
    def add(self, x):
        print(f"adding {x!r}")
        super().add(x)
```

```
s = LoggingSet()
s.add(1)
print(s.contains(1))
```

def p(x):
print("hello"

s.contains =

Method Resolution Order

Search*:

instance

↓

class of instance

↓

base of the class

⋮

WHAT HAPPENS IN THE CASE OF TWO PARENTS

```python
class Encrypted:
    def save(self):
        print("encrypt")
        super().save()

class Compressed:
    def save(self):
        print("compress")
        print("write to disk")

class SecureFile(Encrypted, Compressed):
    pass
```

FACT 1 : Method Resolution Order

instance
↓
class
↓
bases of the class read left to right
⋮

SecureFile().save()

[, Encrypted , Compressed ,]

What is the output of `SecureFile().save()` ?

WHAT HAPPENS IN THE CASE OF TWO PARENTS

'object'

```
class Encrypted:
    def save(self):
        print("encrypt")
        super().save()

class Compressed(Encrypted):
    def save(self):
        print("compress")
        print("write to disk")

class SecureFile(Encrypted, Compressed):
    pass
```

What is the output of `SecureFile().save()` ?

FACT 1 : Method Resolution Order

instance
↓
class
↓
bases of the class read left to right
⋮

FACT 2 :

super() ≠ next class after me in the search order

[instance, class, parent classes left to right, ...,]

EXERCISE: ATTRIBUTE / METHOD LOOKUP

```
class S:
    k = 2
    def h(self, x):
        return T(x + 1)

class T(S):
    p = 8
    def __init__(self, y):
        if y:
            self.k = self.h(y)

class U(T):
    def h(self, x):
        return x
```

EVALUATE

```
s = S()
t = T(0)
t.p = 3
```

ANSWER

(a) `U(5).p` what is it?

(b) `s.k == U.k` — True or False?

(c) which of these is an **integer**?

`s.h(3)` ; `T(1).k` ; `U(1).k`

A NEW ADT: SEQUENCE

"hello"



Specs: a finite, ordered collection, addressed by position

observer ← `length()` — how many elements

observer ← `get(i)` — the element at position i

mutator ← `set(i, x)` — overwrite position i

mutator ← `insert(i, x)` — put x at i, sliding the rest right

mutator ← `delete(i)` — remove at i, sliding the rest left

pre:

post:

A NEW ADT: SEQUENCE

Specs: a finite, ordered collection, addressed by position

```
length()        how many elements
get(i)          the element at position i
set(i, x)       overwrite position i
insert(i, x)    put x at i, sliding the rest right
delete(i)       remove at i, sliding the rest left
```

How do we implement this ADT?

FIXED - SIZE ARRAY

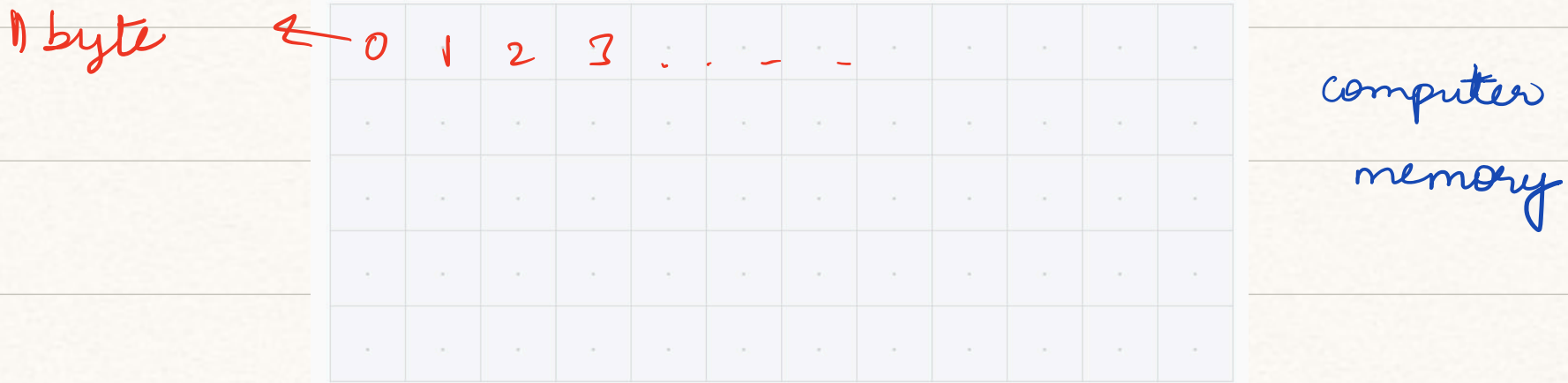
- simplest sequence machine hands you
- one contiguous run of memory
- cut into equal-size slots, one element per slot

FIXED - SIZE ARRAY

- simplest sequence machine hands you
- one contiguous run of memory
- cut into equal-size slots, one element per slot

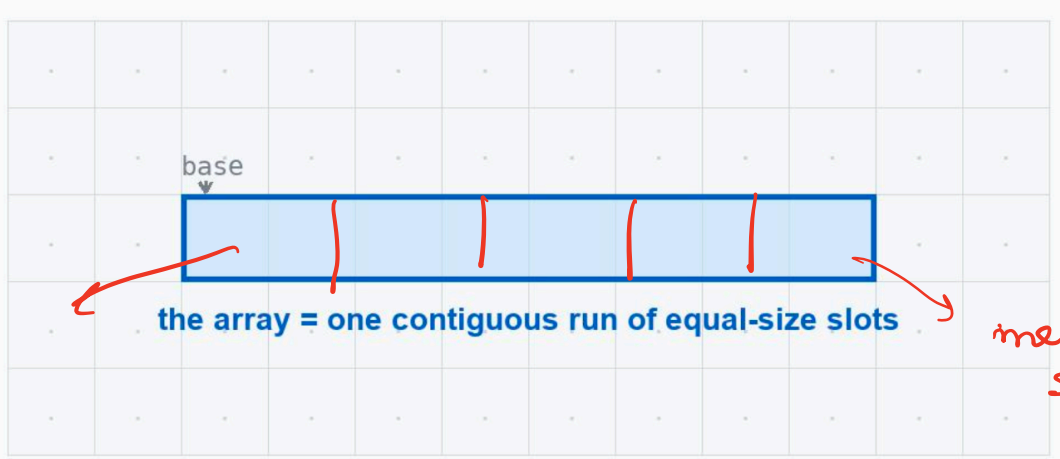
FIXED - SIZE ARRAY

- simplest sequence machine hands you
- one contiguous run of memory
- cut into equal-size slots, one element per slot



FIXED - SIZE ARRAY

- simplest sequence machine hands you
- one contiguous run of memory
- cut into equal-size slots, one element per slot



computer memory

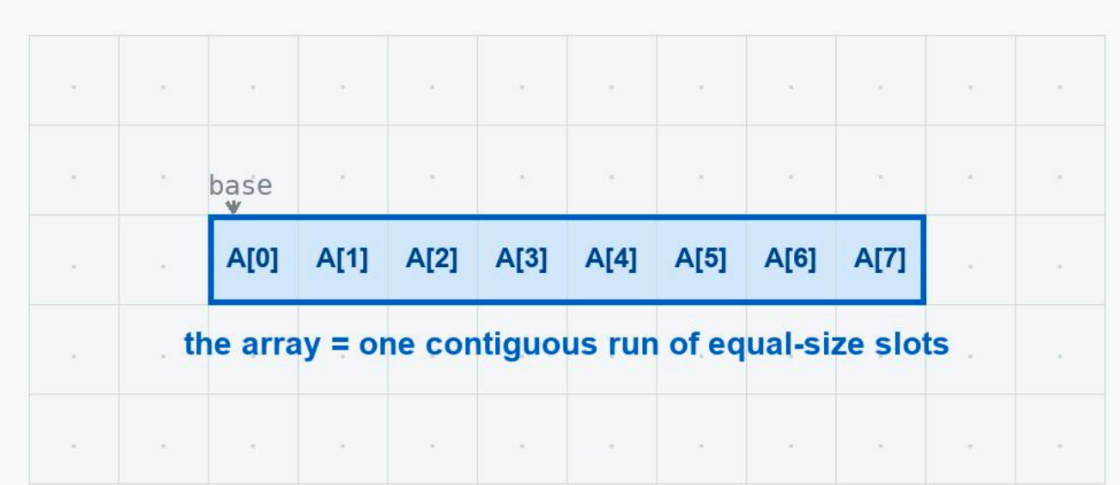
4 bytes

memory size

slot size

FIXED - SIZE ARRAY

- simplest sequence machine hands you
- one contiguous run of memory
- cut into equal-size slots, one element per slot



computer memory

INDEXING IS INSTANT

address (i) = base address + i x slot size

slot i is computed, not searched

O(1) operation

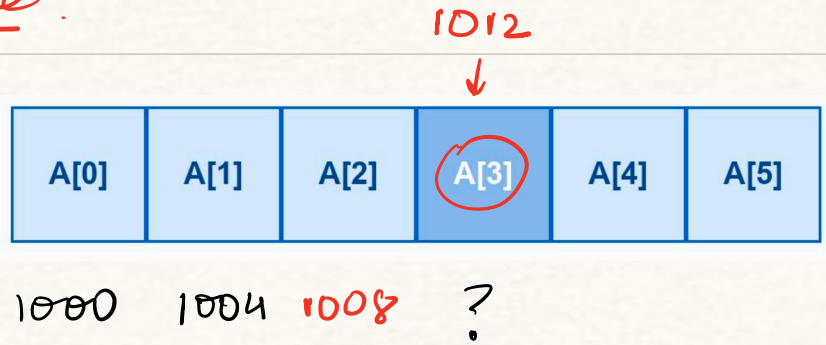
INDEXING IS INSTANT

address(i) = base address + i x slot size

slot i is computed, not searched

address(3) = 1000 + 3 x 4

Example:



What is the time complexity of get(i) and set(i,x) using arrays?

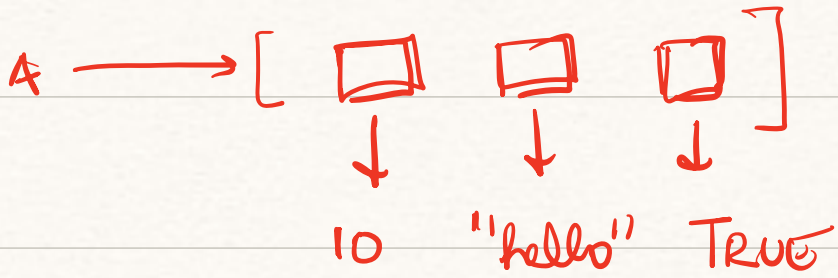
get(i): O(1)

set(i,x): O(1)

NOTE: PYTHON LIST IS ALSO AN ARRAY

But python lists can store arbitrary objects, how?

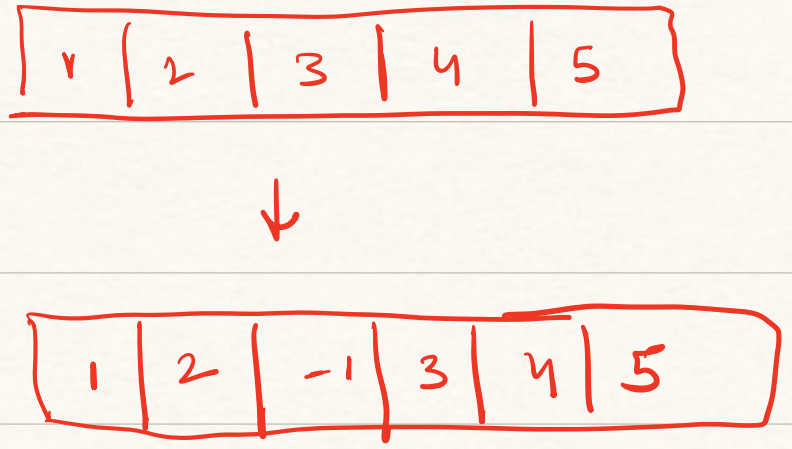
A = [10, "hello", TRUE]



WHAT ABOUT INSERT OPERATION ?

insert (i, x) : insert x at position i

position 2, -1



① - For each element at pos $\geq i$ → move element to the right by one slot

WHAT ABOUT INSERT OPERATION ?

insert (i, x) : insert x at position i

LIMITATION : contiguous memory allocated is fixed.

Two cases

WHAT ABOUT INSERT OPERATION ?

insert (i, x) : insert x at position i

LIMITATION : contiguous memory allocated is fixed.

Two cases

CASE 1 : there is room to add a new element

slide everything from i to right by one slot.

WHAT ABOUT INSERT OPERATION ?

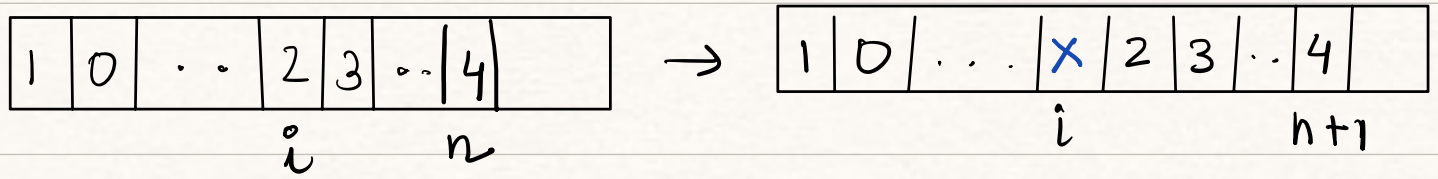
insert (i, x) : insert x at position i

LIMITATION : contiguous memory allocated is fixed.

Two cases

CASE 1 : there is room to add a new element

slide everything from i to right by one slot.



WHAT ABOUT INSERT OPERATION ?

insert (i, x) : insert x at position i

LIMITATION : contiguous memory allocated is fixed.

Two cases

CASE 1 : there is room to add a new element

Slide everything from i to right by one slot.

$$O(n-i)$$

Best case : $O(1)$, add element at last

Worst case : $O(n)$, add to the beginning

WHAT ABOUT INSERT OPERATION ?

insert (i,x) : insert x at position i

LIMITATION : contiguous memory allocated is fixed.

Two cases

CASE 1 : there is room to add a new element

→ slide everything from i to right by one slot.

CASE 2 : array is full !

what to do ?

WHAT ABOUT INSERT OPERATION ?

insert (i,x) : insert x at position i

LIMITATION : contiguous memory allocated is fixed.

Two cases

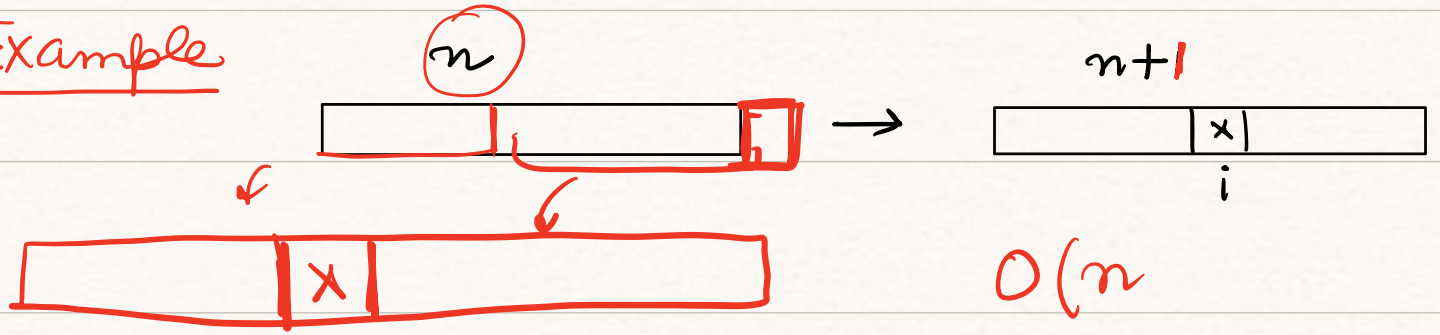
CASE 1 : there is room to add a new element

→ slide everything from i to right by one slot.

CASE 2 : array is full !

grow the array

Example



O(n

COST OF GROWTH

insert (i,x): insert x at position i

Q: grow by exactly **one** each append; building an array of n from empty. Total cost?

$$1+2+3+4+\ldots n = \frac{n \times n+1}{2}$$

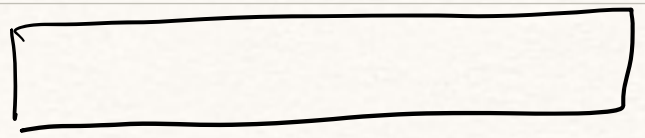
↳ $O(n^2)$

Can we do anything faster?

CAN WE DO BETTER?

Pre-allocate

1M size array

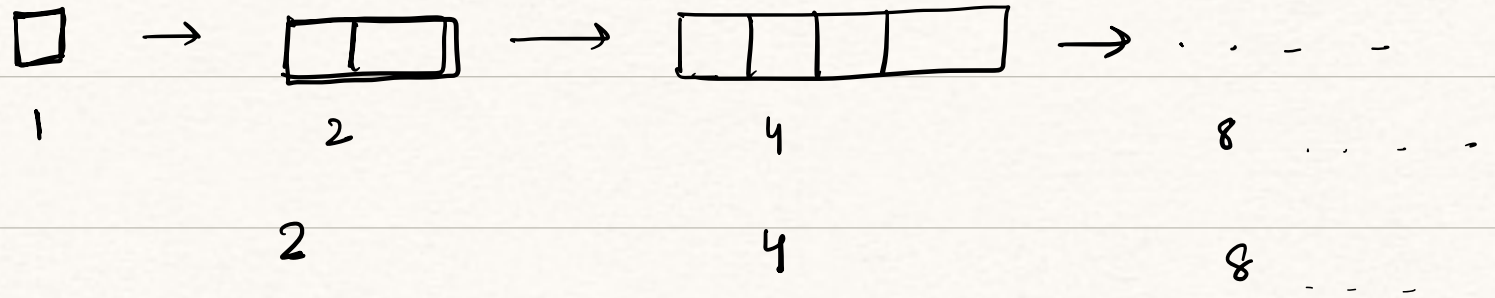


How to smartly grow the array?

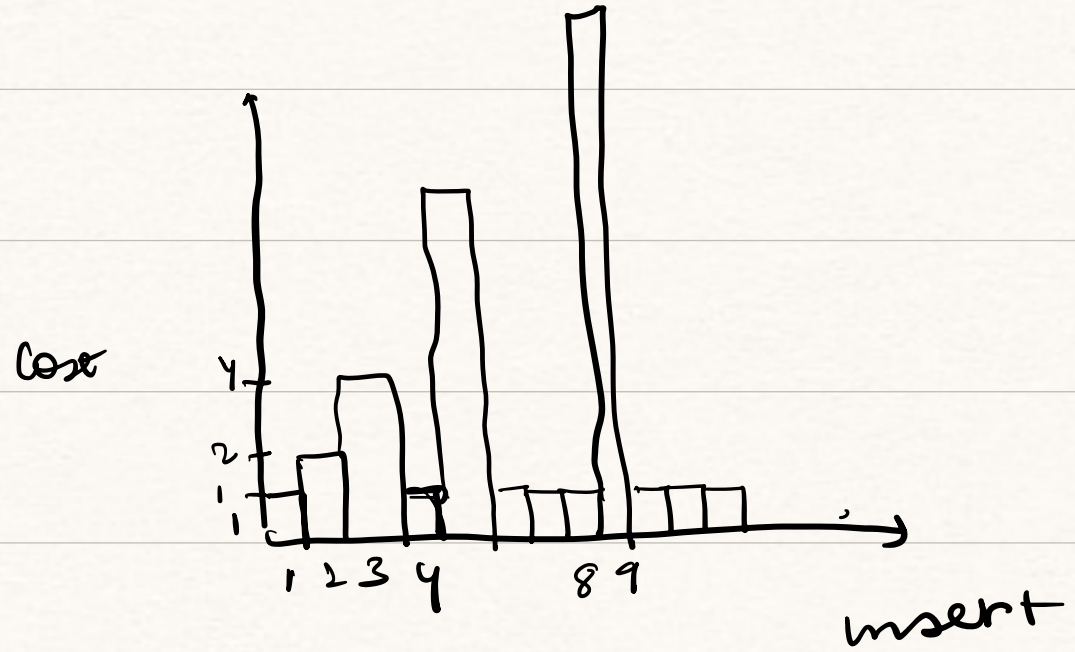
↳ grow by a factor

CAN WE DO BETTER?

Grow by a factor (e.g. 2)



AVERAGE COST OVER n INSERTS: AMORTIZED ANALYSIS



→ resizing : $1 + 2 + 4 + \ldots$ (n) insert operation

$2n - 1$

↳ $O(n)$

→ insert : $O(1)$

amortized resizing cost : $O(1)$

FINAL COST OF INSERTION

insert (i, x) has two costs

- shift after i : $O(n-i)$
- resize if full : $O(1)^*$

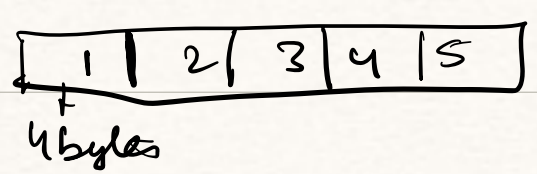
* amortized

Final cost : $O(n-i) + O(1)$
amortized is $O(n-i)$

SIDENOTE: PYTHON ALSO SUPPORTS ARRAY OF VALUES, NOT REFERENCES

- array
- numpy

```
a = array.array('i', [1,2,3,4,5])
a.append(4)
a.append(2.5)
```

array(

)

SEQUENCE ADT USING ARRAYS

Operation	Cost
length()	O(1)
get(i)	O(1)
set(i, x)	O(1)
insert(i, x)	O(n-i)*
remove(i, x)	O(n-i)*

* amortized